

Računarska vežba 11

PYTHON ZADACI ZA KOLOKVIJUM

- 1) Šta je rezultat izvršavanja datog programskog koda? **Detaljno objasniti korake izvršavanja koda.**

```
1  def printArr(arr, index):
2      if(index < len(arr)):
3          print(arr[index])
4          printArr(arr, index+1)
5
6  arr = [1, 3, 5, 7]
7  printArr(arr, 0)
```

Odgovor:

Rezultat izvršavanja programskog koda biće:

```
1
3
5
7
```

Način izvršavanja koda:

Izvršavanje funkcije `printArr`:

- Argumenti funkcije su niz `arr` i `index` (indeks niza)
- Osnovni uslov `if(index < len(arr))`: proverava se da li se prolazi kroz ceo niz. Kada je `index == len(arr)` uslov je `FALSE` i rekurzija se zaustavlja, nema dodatnih poziva funkcije
- Akcija: Kada je uslov `TRUE`, štampa se tekući element `arr[index]`
- Korak rekurzije: Poziv funkcije se vrši za sledeći element niza sa indeksom `index+1` ili `printArr(arr, index+1)`.

Korak 1:

Poziv `printArr([1,3,5,7],0)`

- Provera `0<4` ► TRUE
- Print:1
- Rekurzija: `printArr([1,3,5,7],1)`

Korak 2:

Poziv `printArr([1,3,5,7],1)`

- Provera `1<4` ► TRUE
- Print:3
- Rekurzija: `printArr([1,3,5,7],2)`

Korak 3:

Poziv `printArr([1,3,5,7],2)`

- Provera `2<4` ► TRUE
- Print:5
- Rekurzija: `printArr([1,3,5,7],3)`

Korak 4:

Poziv `printArr([1,3,5,7],3)`

- Provera `3<4` ► TRUE
- Print:7
- Rekurzija: `printArr([1,3,5,7],4)`

Korak 5:

Poziv `printArr([1,3,5,7],4)`

- Provera `4<4` ► FALSE
- Print:7
- STOP: vraćanje bez print

IZLAZ: Sekvenca 1,3,5,7 (svaki broj u novoj liniji)

Šta treba izmeniti u kodu da bi ispis bio u obrnutom redosledu, tj. da bude

● 7
5
3
1

2) Šta je rezultat izvršavanja datog programskog koda? **Objasniti korake izvršavanja koda.**

```
1 def tuple_sum(t):
2     return t[0][0] + t[1][1]
3
4 data = ([10, 20], (30, 40))
5 result = tuple_sum(data)
6 print(result)
```

Odgovor:

Rezultat izvršavanja programskog koda biće:

50

Način izvršavanja koda:

`data = ([10,20], (30,40))`

Promenljiva `data` je struktura torke u kojoj je:

- Prvi element lista `data[0] = [10,20]`
- Drugi element toraka `data[1] =(30,40)`

`t[0][0]` ► označava pristup prvom elementu liste tj. broju 10

`t[1][1]` ► označava drugom elementu torke tj. broju 40

Funkcija `tuple_sum(t)` vraća vrednost:

`t[0][0]+ t[1][1] = 10 + 40 = 50`

3) Šta je rezultat izvršavanja datog programskog koda?

Objasniti korake izvršavanja uslovne petlje **if-elif-else**.

```
1 x = 0
2 a = 5
3 b = 5
4
5 if a > 0:
6     if b < 0:
7         x = x + 5
8     elif a > 5:
9         x = x + 4
10    else:
11        x = x + 3
12 else:
13     x = x + 2
14
15 print(x)
```

Odgovor:

Rezultat izvršavanja programskog koda biće:

3

Način izvršavanja koda:

Izvršavanje programskog koda petlje...

```
1  x = 0
2  a = 5
3  b = 5
4
5  if a > 0:                # True → a je 5
6      if b < 0:            # False → b je 5
7          x = x + 5
8      elif a > 5:         # False → a nije veće od 5
9          x = x + 4
10     else:               # izvršava se ovaj blok
11         x = x + 3
12 else:
13     x = x + 2
14
15 print(x)
```

Prvi uslov **if**: `a>0`, **TRUE** prelazi se na sledeću liniju koda

Drugi uslov **if**: `b<0`, **FALSE** prelazi se na **elif**

Treći uslov **elif**: `a>5`, **FALSE** pa se izvršava `x=x+3` u okviru **else**

- 4) Napisati program koji od korisnika traži da unese string. Program treba da kreira novi string od unetog stringa tako da se drugi karakter unetog stringa zameni sa zvezdicom i da se na kraj stringa dodaju tri znaka uzvika. Na kraju treba štampati novi string. Tipičan izlaz može da izgleda ovako:

```
Unesite string: Australija
A*straliija!!!
```

Moguće rešenje:

```
s=input('Unesite string: ')
s=s[0]+'*'+s[2:]+ '!!!'
print(s)
```

- 5) Koristeći programski kod za algoritam LINEARNOG PRETRAŽIVANJA (koji je rađen na vežbama), napisati program koji će vršiti pretraživanje broja u nizu ali samo ZA PRVU POLOVINU NIZA kao npr:

- Niz: [9, 3, 7, 1, 5, 8, 2, 6]
Pretraga broja 7: 2
Pretraga broja 5: -1

(Broj 7 je pronađen na indeksu 2, broj 5 je pronađen u drugoj polovini niza pa je -1, tj. nije pronađen!!!)

Moguće rešenje:

```
1 def linear_search_first_half(arr, target):
2     n = len(arr)
3     half = n // 2    # sredina niza
4
5     # Pretraga samo u prvoj polovini niza
6     for i in range(half):
7         if arr[i] == target:
8             return i    # return index ako je pronađen
9
10    return -1    # nije pronađen
11
12
13 # Unos niza
14 data = [9, 3, 7, 1, 5, 8, 2, 6]
15 print("Niz:", data)
16
17 print("Pretraga broja 7:", linear_search_first_half(data, 7))
18 print("Pretraga broja 5:", linear_search_first_half(data, 5))
19
```

- 6) Za dati binarni niz arr[] veličine n, koji je sortiran u nerastućem rasporedu, prebrojati broj jedinica. Zadatak uraditi upotrebom niza.

```
def countOnes(arr):
    count = 0
    for num in arr:
        if num == 1:
            count += 1
        else:
            break
    return count

arr = [1, 1, 1, 1, 0, 0, 0]
print(countOnes(arr))
```

Output

4

- 7) Napisati program koji generiše 100 slučajnih brojeva koji su ili 0 ili 1. Zatim odredite najduži uzastopni niz nula. Na primer, najduži uzastopni niz nula u listi [1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0] je 4.

```
from random import randint

L=[randint(0,1) for i in range(100)]

LB =[]

i = 0
while i < len(L):
    if L[i] == 0:
        j = i
        br = 0
        while j < len(L):
            br = br + 1
            if L[j] != 0:
                break
            j = j + 1
        LB.append(br)
    i = i +1
print(L)
print('Trazeni rezultat je:', max(LB))
```

8)

- 8) Napisati program koji izračunava sumu $1-2+3-4+ \dots +1999 - 2000$

```
suma=0
for i in range(1,2001):
    if i%2==1: #ako je broj i neparan
        suma=suma+i
    else:
        suma=suma-i
print(suma)
```

9)

- 9) Šta je rezultat izvršavanja datog programskog koda? **Objasniti korake izvršavanja koda.**

```
1  nums = [55, 20, 85, 10, 40]
2  r = 0
3  for i in range(len(nums)):
4      r += nums[i] - i
5  print(r)
```

Odgovor:

Rezultat izvršavanja programskog koda biće:

200

Način izvršavanja koda:

Korak 1: Inicijalizacija promenljivih

Prvo se inicijalizuje lista `nums` i promenljiva `r`

```
nums=[55, 20, 85, 10, 40]
r=0
```

Korak 2: Petlja kroz listu

Petlja `for` iterira kroz `nums` listu, dužina `nums` liste je `5`, pa će petlja imati iteracije `i=0, 1, 2, 3, 4, 5`

Korak 3: Računanje vrednosti za `r` za svaku iteraciju

Linijom `r += nums[i] - i` se izračunava vrednost `r`

```
Za i=0: r=0+nums[0]-0=0+55-0=55
Za i=1: r=55+nums[1]-1=55+20-1=74
Za i=2: r=74+nums[2]-2=74+85-2=157
Za i=3: r=157+nums[3]-3=157+10-3=164
Za i=4: r=164+nums[4]-4=164+40-4=200
```

10) Šta je rezultat izvršavanja datog programskog koda? **Objasniti korake izvršavanja koda.**

```
1  lst = [10, 20, 30, 40]
2  result = 0
3  for i in lst:
4      for j in lst:
5          result += (i == j)
6  print(result)
```

Odgovor:

Rezultat izvršavanja programskog koda biće:

4

Način izvršavanja koda:

Korak 1: Inicijalizacija promenljivih

Prvo se inicijalizuje lista `lst` i promenljiva `result`

Korak 2: Dvostruka petlja `for`

Rezultat je poređenje indeksa `i` spoljašnje i indeksa `j` unutrašnje `for` petlje

	1	2	3	4
1	✓	✗	✗	✗
2	✗	✓	✗	✗
3	✗	✗	✓	✗
4	✗	✗	✗	✓

Prikaz indeksa, indeksi redova su spoljašnja `for` petlja (`i`), kolone su indeksi unutrašnje `for` petlje (`j`):

- ✓ predstavlja `i==j` (`True` → dodaje +1 u `result`)
- ✗ predstavlja `i!=j` (`False` → dodaje 0 u `result`)

Dijagonalno su predstavljeni slučajevi za `i, j` indekse `1==1, 2==2, 3==3, 4==4`